Colab과 Python 기초

학습목표

1) Colab에서 노트북을 만들고 저장·실행할 수 있다

코드 셀과 텍스트 셀, 실행 순서, 오류 메시지 읽는 법

2) 변수에 값을 담고 자료형을 구분한다

정수·실수·문자열·불리언·None, type()으로 확인

3) 산술·비교·논리 연산으로 계산하고 판단한다

우선순위, round·divmod, and·or·not, in

4) 문자열과 리스트를 다룬다

인덱싱·슬라이싱, 문자열 메서드, 리스트 추가·삭제·정렬

DAY 1의 흐름

노트북 0절

Colab 기초

셀 종류, 단축키, 저장, 실행 순서, 오류 읽기

노트북 1~2절

변수와 자료형

변수, f-string, 객체, None, 함수와 메서드

노트북 3~4절

연산자

산술·우선순위·round·divmod, 비교·논리·in

노트북 5~6절

문자열과 리스트

인덱싱·슬라이싱, 메서드, 추가·삭제·정렬

노트북 7절

학습 점검

확인문제와 정답

Colab 기초

Python 이란

Python 은 간결하고 읽기 쉬운 문법을 가진 범용 프로그래밍 언어다. 데이터 분석, 인공지능, 웹 개발, 자동화 등 넓은 분야에서 쓰인다.

1) 읽기 쉬운 문법

영어 문장에 가까워 비전공자가 시작하기 좋다

2) 들여쓰기로 블록 표현

```
if sales >= 100:
    print("기준 이상")
```

3) 운영체제를 가리지 않는다

Windows·macOS·Linux 어디서나 돌아간다. 설치 없이 브라우저(Colab)에서도 쓸 수 있다

4) 큰 생태계

표준 라이브러리와 수많은 외부 패키지

5) 오픈 소스

무료로 쓰고, 누구나 내부를 볼 수 있다

6) 활용

데이터 분석·시각화, 머신러닝·딥러닝, 웹, 데이터베이스, 자동화

Python 3.13 공식 문서(한국어): https://docs.python.org/ko/3.13/

개발 도구: Google Colab

Google Colab

브라우저에서 쓰는 호스팅형 Jupyter Notebook

호스팅형: 내 컴퓨터가 아니라 구글 서버에서 실행된다 · Jupyter Notebook: 코드, 실행 결과, 설명 글을 한 문서에 담는 형식

- 설치 없이 구글 계정으로 바로 시작
- 무료, GPU·TPU 제공
- 실행 시간·자원은 이용 상황에 따라 달라진다

브라우저
설치 없이 접속

→

구글 계정
로그인

→

Colab 노트북
코드 실행 · 저장(Drive)

Google Colab: https://colab.research.google.com/ · Colab FAQ: https://research.google.com/colaboratory/faq.html

데이터 분석 라이브러리

NumPy

수치 연산의 기반 · 배열 단위 계산

예: 센서 값 1,000개에 보정값을 한 번에 곱하기

pandas

표 형태 데이터 · 불러오기·정리·집계

예: 엑셀 매출표를 읽어 지역별 합계 내기

matplotlib · seaborn

그래프 · 집계 결과를 시각화

예: 월별 생산량을 선 그래프로 그리기

scipy.stats

확률분포와 통계 함수 · 검정

예: 두 라인의 불량률 차이 검정하기

scikit-learn

머신러닝

예: 센서 값으로 설비 고장 예측하기

```
# 라이브러리는 import 로 불러온다
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

Python 3.13 공식 문서(한국어): https://docs.python.org/ko/3.13/ · 모듈(한국어): https://docs.python.org/ko/3.13/tutorial/modules.html

Colab 접속과 새 노트북 만들기

1) 구글 계정으로 로그인한다

Colab 은 구글 계정과 Google Drive 를 사용한다

2) colab.research.google.com 에 접속한다

시작 화면이 열리면 최근 노트북 목록이 보인다

3) 파일 → 새 노트북을 선택한다

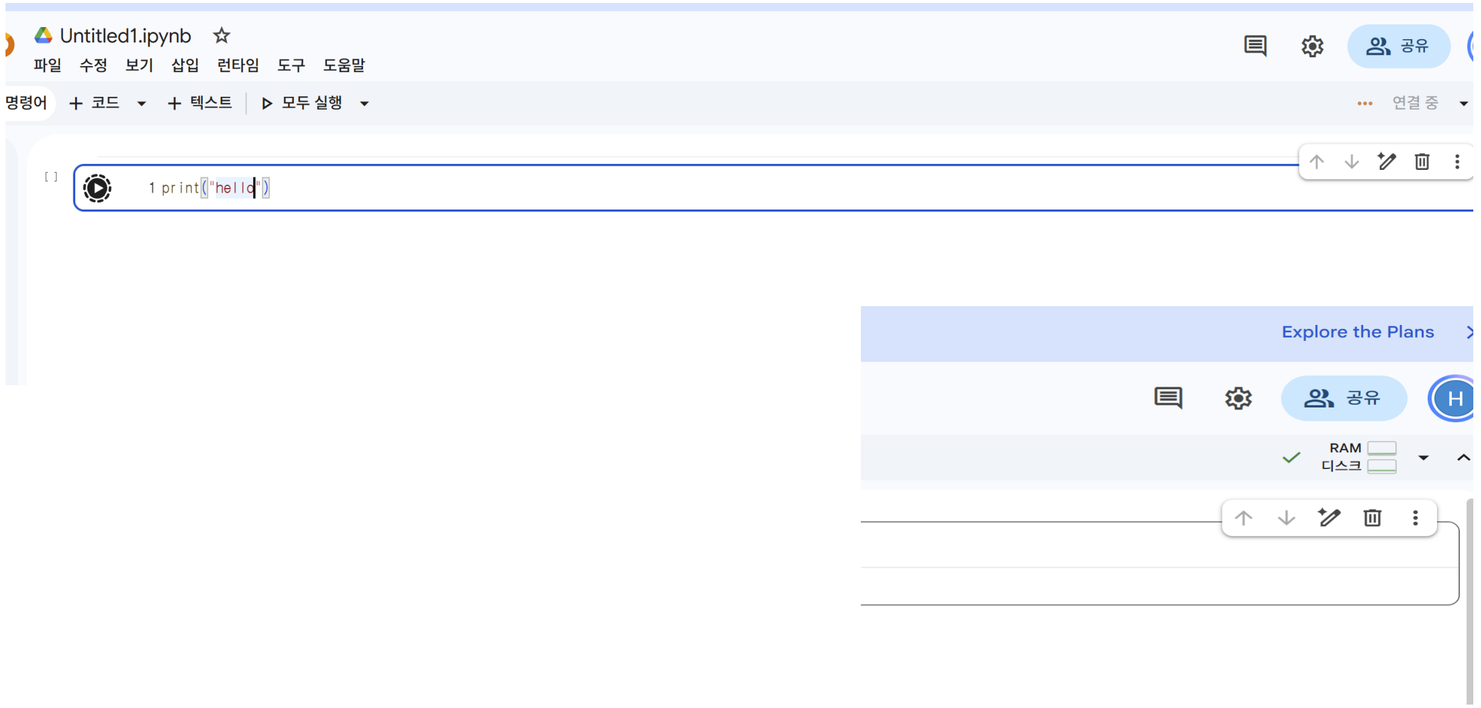
화면 언어와 업데이트에 따라 메뉴 이름이 조금 다를 수 있음



Google Colab: https://colab.research.google.com/

코드 실행과 버전 확인

코드 셀에 입력한 뒤 Shift + Enter 로 실행한다. 처음 실행하면 '연결' 과정이 진행되어 RAM · 디스크를 할당받는다.




```
print("Hello")
import sys
print(sys.version)
print(sys.executable)
```

`sys.version` → 3.13.x (Python 버전)

`sys.executable` → /usr/bin/python3 (실행 경로)

실제 값은 현재 런타임에 따라 다르다

RAM: 실행 중인 변수와 데이터를 올려 두는 작업용 메모리
디스크: 파일 저장 공간. 런타임이 끝나면 지워지므로 남길 파일은 Drive 에 저장한다

sys.version(한국어): https://docs.python.org/ko/3.13/library/sys.html#sys.version

파일 이름 변경과 저장

1) 파일 이름을 바꾼다

상단의 Untitled0.ipynb 를 클릭해 DAY1_Python_기초 처럼 바꾼다

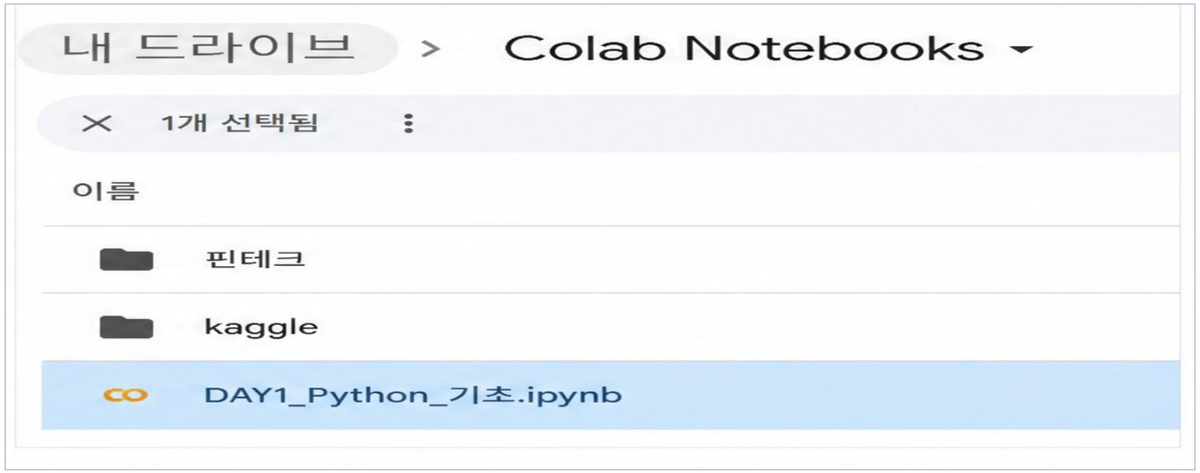


2) 드라이브에 저장한다

파일 → 저장. 이미 드라이브 문서인지에 따라 메뉴 동작이 다를 수 있다

3) 저장 위치를 확인한다

Google Drive 의 Colab Notebooks 폴더에 보관된다



Google Colab: https://colab.research.google.com/

Google Drive 와 Colab 연결

드라이브에 저장된 노트북 열기

1) Google Drive 에서 .ipynb 파일을 마우스 오른쪽 버튼으로 클릭한다
2) 연결 앱 → Google Colaboratory 를 선택한다

Google Colaboratory 가 연결 앱에 없을 때

1) 새로 만들기 → 더보기 → 연결할 앱 더보기
2) Google Colaboratory 를 검색해 연결한다

Google Colab: https://colab.research.google.com/

런타임 유형 확인

1) 표준 런타임 (CPU)

일반적인 Python 코드, 데이터 전처리, pandas·NumPy 기반 분석에 사용한다. 오늘 실습은 모두 이 환경이면 충분하다.

2) GPU · TPU 런타임

GPU: 그래픽 처리 장치 (T4, L4, A100 등)

같은 계산을 한꺼번에 많이(병렬로) 처리해서 대규모 행렬 연산에 쓴다. 딥러닝 학습·추론(PyTorch, TensorFlow), 이미지 처리에 적합하다.

TPU (Tensor Processing Unit)

구글이 머신러닝 전용으로 설계한 가속기다. 매우 큰 모델의 학습에 특화되어 있다. TensorFlow, JAX, PyTorch/XLA 처럼 TPU 를 지원하는 라이브러리에서만 쓸 수 있다.

이미지나 음성 데이터라고 항상 GPU 가 필요한 것은 아니고, 표 데이터라도 대규모 딥러닝이면 GPU 를 쓴다. 데이터 종류가 아니라 계산량이 기준이다

Colab FAQ: https://research.google.com/colaboratory/faq.html

런타임 유형 변경 방법

변경 방법

런타임 → 런타임 유형 변경 → 하드웨어 가속기 선택.

이 과정에서는 '없음(None)' 또는 CPU 그대로 둔다.



Colab FAQ: https://research.google.com/colaboratory/faq.html

마크다운으로 문서화하기

1) + 코드 / + 텍스트

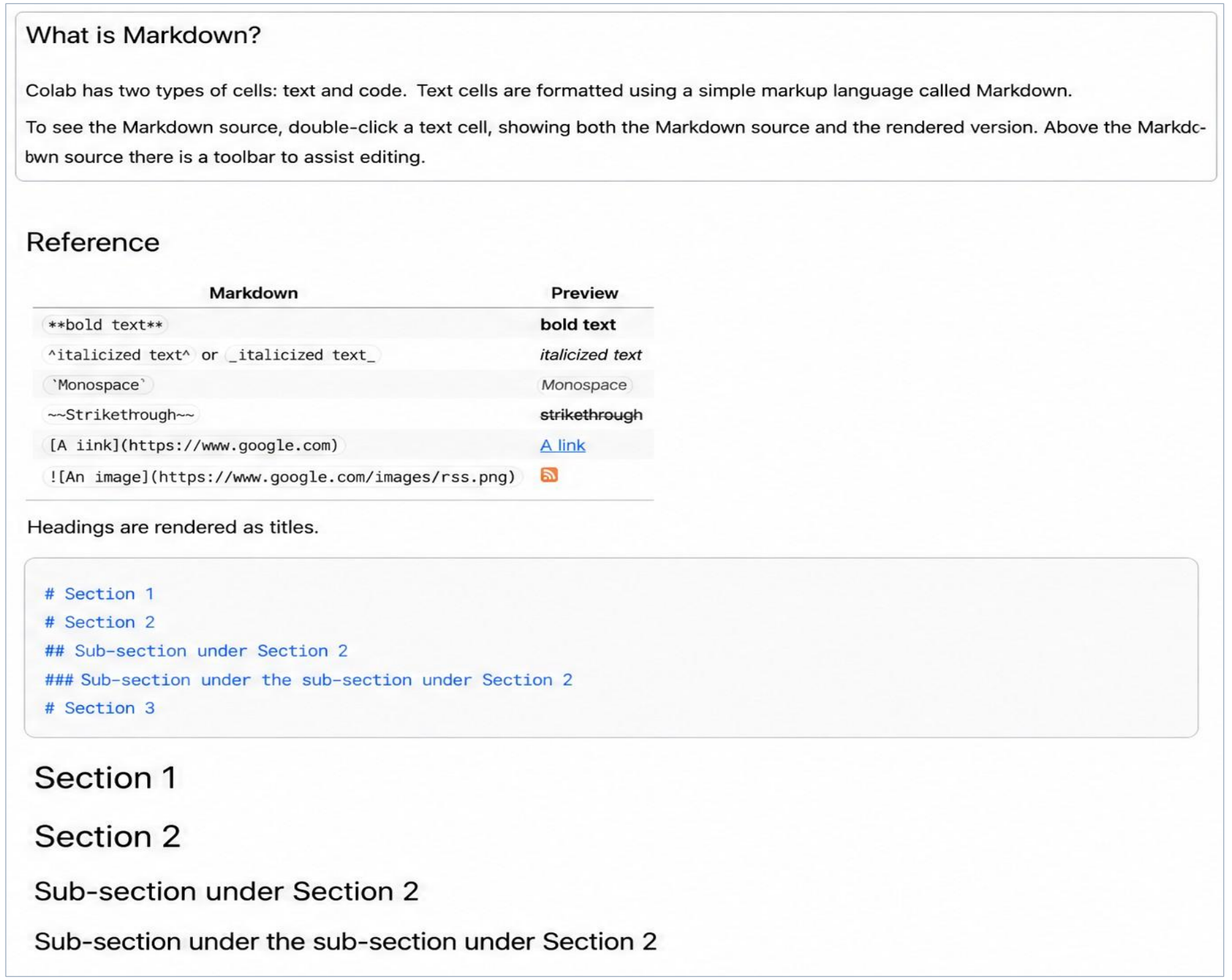
코드 셀과 텍스트 셀을 추가한다. 텍스트 셀에는 마크다운을 쓴다

2) 마크다운 기본

제목, ## 소제목, - 목록, **굵게** 입력하면 오른쪽에 미리보기가 보인다

3) 코드 주석

Python 코드에서 # 뒤는 주석이다. 실행되지 않고 설명으로 남는다



Colab 마크다운 안내: https://colab.research.google.com/notebooks/markdown_guide.ipynb?hl=ko

실행 순서와 단축키

실행 순서가 중요한 이유

Colab 은 위에서 아래로 자동 실행하지 않는다. 변수를 만든 셀을 먼저 실행해야 뒤의 셀에서 쓸 수 있다.

```
total_sales = 350000
# 먼저 실행
print(total_sales)
# 그 다음
```

셀 왼쪽 번호 [1], [2] 가 실제 실행 순서다.
순서가 꼬이면 런타임 → 세션 다시 시작 및 모두 실행.

순서가 꼬이면: 아래 셀을 먼저 실행한 경우

```
[2]   total_sales = 350000
```

```
[1]   print(total_sales)          → NameError
```

자주 쓰는 단축키 (Mac 은 Ctrl 대신 Cmd)

동작	Windows	Mac
현재 셀 실행	`Ctrl + Enter`	`Cmd + Enter`
실행하고 다음 셀로	`Shift + Enter`	`Shift + Enter`
위 / 아래에 셀 추가	`Ctrl + M, A / B`	`Cmd + M, A / B`
셀 삭제	`Ctrl + M, D`	`Cmd + M, D`
삭제한 셀 복구	`Ctrl + M, Z`	`Cmd + M, Z`
주석 켜고 끄기	`Ctrl + /`	`Cmd + /`
단축키 전체 보기	`Ctrl + M, H`	`Cmd + M, H`

Python 오류와 예외(한국어): https://docs.python.org/ko/3.13/tutorial/errors.html

주석

셀 안의 글이 모두 실행되는 것은 아니다. 주석은 건너뛰고, 마지막 줄의 값은 자동으로 보여 준다.

주석: 실행되지 않는 설명

#

```
sales = 120000   # 1월 매출
```

```
sales = 120000
tax = sales * 0.1
tax
# 12000.0 이 그대로 출력된다
```

중간 줄의 값을 보려면 print()가 필요하다. 마지막 줄만 자동으로 보인다

Python 자습서 · 주석(한국어): https://docs.python.org/ko/3.13/tutorial/introduction.html

오류 메시지 · 실제 출력으로 읽기

실행한 셀

```
total_sales = 350000
print(total_sale)      # s 를 빠뜨렸다
```

출력 (모양은 Colab 버전에 따라 조금 다르다)

```
NameError        Traceback (most recent call last)                    2
ipython-input-2 in <cell line: 0>()
      1 total_sales = 350000
----> 2 print(total_sale)                                             3

NameError: name 'total_sale' is not defined                           1
```

1 마지막 줄을 읽는다

name 'total_sale' is not defined: total_sale 이라는 이름이 없다

2 오류 이름을 확인한다

NameError: 정의하지 않은 이름을 썼다 → 철자와 실행 순서를 본다

3 화살표 줄을 찾는다

----> 2: 이 셀의 2번째 줄에서 났다

원인: total_sales 를 total_sale 로 잘못 적었다. 철자를 고치면 350000 이 출력된다

Python 오류와 예외(한국어): https://docs.python.org/ko/3.13/tutorial/errors.html

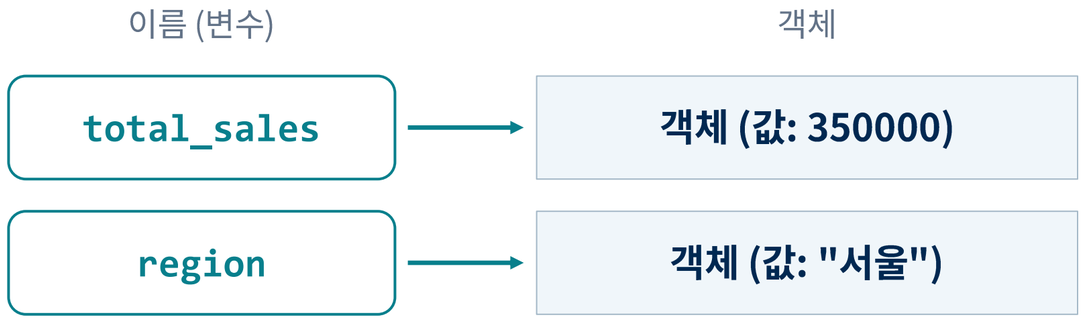
변수와 자료형

변수란

변수는 객체를 가리키는 이름이다. 변수에 값을 대입할 때는 등호(=)를 쓴다

객체는 값과 자료형을 함께 가진 데이터다

1) 변수는 객체를 가리킨다



코드에 total_sales 라고 쓰면 그 객체의 값 350000 을 가져와 사용한다

2) 대입(할당)문 읽는 순서

```
total = 100 + 250
```

1단계 오른쪽을 먼저 계산한다: 100 + 250 → 350

2단계 왼쪽 변수에 대입한다: total → 350

= 는 "같다"가 아니라 "대입한다"는 뜻이다. 같은지 비교할 때는 == 를 쓴다

3) 변수에 담는 이유

```
print(350000 * 0.1)
```

숫자만 있어서 무슨 계산인지 알 수 없다

```
print(total_sales * tax_rate)
```

이름만 읽어도 "매출 × 세율" 이라는 뜻이 보인다

객체, 값, 형(한국어): https://docs.python.org/ko/3.13/reference/datamodel.html#objects-values-and-types

변수명 규칙

사용할 수 있는 변수명

total_sales 영문 소문자 + 밑줄
sales_2026 숫자는 뒤에
region 뜻이 드러나는 이름

잘못된 변수명 (오류)

2026_sales 숫자로 시작
total sales 띄어쓰기
if, for 예약어

변수명 규칙 (주의)

1) 스네이크 케이스(snake_case)로 의미가 드러나게 짓는다 (예: total_sales)
2) 식별자는 숫자로 시작할 수 없다
3) 글자·숫자·밑줄만 쓴다. 한글도 쓸 수 있지만 관례상 영문 소문자를 쓴다
4) 대소문자를 구분한다 (case sensitive)
5) 예약어(if, for 등)를 쓰면 구문 오류(SyntaxError)가 발생한다
6) 내장 함수 이름(print, sum 등)은 재할당되지만 섀도잉으로 원래 기능을 가리므로 피한다

식별자(한국어): https://docs.python.org/ko/3.13/reference/lexical_analysis.html#identifiers

f-string 형식

출력 형식은 f-string 안에서 바로 정한다. 중괄호 { } 안에 변수를 넣고, : 뒤에 모양을 적는다.

f-string	결과	뜻
`f"{total:,}"`	1,200,000	천 단위 쉼표
`f"{rate:.1f}"`	3.5	소수 첫째 자리까지
`f"{name}의 매출"`	서울의 매출	변수 끼워 넣기

```
total = 1200000
rate = 3.46
name = "서울"
print(f"{name}의 매출: {total:,}원, 증가율 {rate:.1f}%")  # 서울의 매출: 1,200,000원, 증가율 3.5%
```

값 자체는 바뀌지 않고 보이는 모양만 바뀐다

포맷 문자열 리터럴(한국어): https://docs.python.org/ko/3.13/tutorial/inputoutput.html#formatted-string-literals

객체와 자료형

개념

- 객체는 식별성(Identity), 자료형(Type), 값(Value)을 가지는 데이터의 단위이며, 자료형에 따라 사용할 수 있는 연산과 메서드가 결정된다.
- Python 의 모든 값은 객체(object)이고 자료형(type)을 가진다
- 정수 int, 실수 float, 문자열 str, 참·거짓 bool, 값 없음 None
- 자료형은 type() 으로 확인한다
- 메서드는 객체 뒤에 점(.)을 찍어 호출한다 (예: x.upper())
- None 은 0 이나 빈 문자열이 아니라 '값이 없음'이다



객체 = 데이터와 메서드가 하나로 묶인 단위
`name.upper()` 를 실행하면 '제품A' 가 나온다
점(.)은 객체의 메서드에 접근하는 문법이다

Python 내장 자료형(한국어): https://docs.python.org/ko/3.13/library/stdtypes.html

객체의 세 가지 특징: 값 · 자료형 · 식별성

객체가 가지는 3가지 특징

1) 값 (value)

객체가 가진 실제 데이터

```
a = [10, 20]
```

2) 자료형 (type)

값의 종류: 정수, 문자열, 리스트 등

```
type(a)  # <class 'list'>
```

3) 식별성 (identity)

두 이름이 같은 객체를 가리키는지 구분

```
a is b  # True
```

함수 · 메서드 · None

type(a): 자료형 확인 | "abc".upper(): 메서드 호출

None: 값이 없음을 나타내며 0이나 빈 문자열과 다름

같은 값 ≠ 같은 객체

```
a = [10, 20]      # 리스트 생성
b = a             # 같은 객체를 참조
c = [10, 20]      # 별도의 리스트 생성
```



비교 결과

`a == c`	`# True`	값이 같다
`a is b`	`# True`	같은 객체를 참조
`a is c`	`# False`	서로 다른 객체

== 는 값이 같은지, is 는 같은 객체인지 비교한다.

객체, 값, 형(한국어): https://docs.python.org/ko/3.13/reference/datamodel.html#objects-values-and-types

객체와 자료형

```
# 자료형 다섯 가지: 모든 값은 자료형을 가진다
count = 12              # int: 정수
rate = 3.2              # float: 실수
name = "제품a"          # str: 문자열
is_pass = True          # bool: 참·거짓
memo = None             # None: 값 없음

print(type(rate))       # 자료형 확인 → <class 'float'>
print(len(name))        # 함수: 독립적으로 호출 → 3
print(name.upper())     # 메서드: 객체 뒤에 점을 찍어 호출 → 제품A
```

Python 내장 자료형(한국어): https://docs.python.org/ko/3.13/library/stdtypes.html · 내장 함수(한국어): https://docs.python.org/ko/3.13/library/functions.html

숫자와 문자열

같은 120 이라도 숫자 120 과 문자열 "120" 은 계산 결과가 다르다.

```
print(120 + 30)            # 150: 숫자 덧셈
print("120" + "30")        # 12030: 문자열 연결
print(int("120") + 30)     # 150: 숫자로 바꾼 뒤 계산
print(float("3.5") + 1)    # 4.5: 소수가 있으면 float
```

숫자: 더한다

120	+	30	=	150

문자열: 이어 붙인다

"120"	+	"30"	=	"12030"

1) type() 으로 확인하기

따옴표로 감싼 값은 문자열이다. CSV 에서 읽은 값도 계산 전에 자료형을 확인한다.

2) 바꿀 수 있는 값인지 확인하기

int("120") 은 가능하지만 int("제품A") 는 ValueError 가 발생한다.

Python 내장 자료형(한국어): https://docs.python.org/ko/3.13/library/stdtypes.html

자료형의 분류

모든 값은 자료형을 가진 객체

숫자형

Numeric Types ↗

정수 (integer)

`int` 소수점 없는 수

실수 (floating point number)

`float` 소수점 있는 수

예: `350000` · `3.8`

텍스트 시퀀스형

Text Sequence Type ↗

문자열 (string)

`str` 글자의 나열
변경 불가 (immutable)

예: `"서울"`

시퀀스형

Sequence Types ↗

리스트 (list)

`list` 순서 있음
변경 가능 (mutable)

튜플 (tuple)

`tuple` 순서 있음
변경 불가 (immutable)

예: `[100, 300]` · `(60, 90)`

매핑형

Mapping Types ↗

딕셔너리 (dictionary)

`dict` 키로 값을 찾음
변경 가능 (mutable)

예: `{"서울": 350000}`

집합형

Set Types ↗

집합 (set)

`set` 중복 없음
순서 없음

예: `{"M01", "M02"}`

불리언형

Boolean Type ↗

불리언 (Boolean)

`bool` True, False 두 값뿐

예: `350000 > 0` → `True`

내장 상수

Built-in Constants ↗

None

`NoneType` None 의 자료형
값이 없음을 나타냄

예: `temperature = None`

내장형(한국어): https://docs.python.org/ko/3.13/library/stdtypes.html · 내장 상수(한국어): https://docs.python.org/ko/3.13/library/constants.html

Python 의 기본 자료형

분류	자료형	이름	예시	용도
숫자형	int	정수	12, -3, 0	개수, 수량
숫자형	float	실수	3.2, 120000.0	비율, 평균, 측정값
텍스트 시퀀스형	str	문자열	"제품A", "서울"	이름, 코드, 텍스트
시퀀스형	list	리스트	[100, 300, 200]	순서 있는 값의 모음 (변경 가능)
시퀀스형	tuple	튜플	(60, 90)	순서 있는 값의 모음 (변경 불가)
매핑형	dict	딕셔너리	{"서울": 500}	키로 값을 찾는 저장
집합형	set	집합	{"M01", "M02"}	중복 없는 값의 모음
불리언형	bool	참·거짓	True, False	조건 판단 결과
내장상수	NoneType	값 없음	None	아직 값이 정해지지 않음

* 공식 분류에서 bool 은 int 의 하위형이다

Python 내장 자료형(한국어): https://docs.python.org/ko/3.13/library/stdtypes.html

None 과 is

"값이 없다"도 값이다. None 은 그것을 나타내는 이름이고, 확인할 때는 is None 으로 쓴다.

None: 값이 없음

"아직 값이 없다"를 나타내는 특별한 객체

0도 아니고 빈 문자열도 아니다

```
temperature = None      # 센서 값이 아직 없음
if temperature is None:
    print("측정 대기")
# 측정 대기
```

== 과 is 의 차이

== 두 값이 같은가 (내용 비교)

is 같은 객체인가

None 비교는 반드시 is 를 쓴다

`a = [1, 2]; b = [1, 2]`

`a == b → True,  a is b → False`

함수 결과가 None 인지 확인할 때 if result is None: 으로 쓴다. == None 은 쓰지 않는다

None(한국어): https://docs.python.org/ko/3.13/library/constants.html#None · PEP 8: https://peps.python.org/pep-0008/

함수와 메서드의 차이

메서드는 객체(값) 안에 들어 있는 함수다. 함수는 이름으로 바로 호출하고, 메서드는 객체 뒤에 점을 찍어 호출한다.

함수: 독립적으로 호출한다

```
sales = [100, 200, 300]; rate = 3.2
len("abc")     # 3
print(sales)   # [100, 200, 300]
type(rate)     # <class 'float'>

# 이름(인자) 형태. 내장 함수(print, len, type)와
# 직접 만든 함수(def)가 있다
```

메서드: 객체에 속한 함수

```
sales = [100, 200]; lot = " A01 "
"abc".upper()  # 'ABC'
sales.append(400)  # [100, 200, 400]
lot.strip()    # 'A01'

# 객체.메서드() 형태. 그 객체의 값을 다루며
# 자료형마다 가진 메서드가 다르다
```

문자열에는 문자열 메서드, 리스트에는 리스트 메서드가 있다.

내장 함수(한국어): https://docs.python.org/ko/3.13/library/functions.html

연산자

숫자형과 산술 연산

산술 연산자 종류

연산자	의미	예시
+	덧셈	7 + 2 → 9
-	뺄셈	7 - 2 → 5
*	곱셈	7 * 2 → 14
/	나눗셈	7 / 2 → 3.5
//	몫	7 // 2 → 3
%	나머지	7 % 2 → 1
**	거듭제곱	7 ** 2 → 49

```
# 산술 연산: 매출 합계와 평균을 계산한다
total = 120000 + 150000 + 90000

# 합계 → 360000
average = total / 3

# 나눗셈의 결과는 항상 float
print(average)             # 120000.0
print(round(3.14159, 2))   # 3.14

# 소수점 둘째 자리까지 반올림
print(7 // 2, 7 % 2)       # 몫과 나머지 → 3 1

# 복합 대입: total = total + 50000
total += 50000
print(total)               # 410000
```

Python 산술 연산(한국어): https://docs.python.org/ko/3.13/tutorial/introduction.html#numbers

round · divmod · math.ceil

함수	하는 일	예	결과
`round(x, n)`	소수 n자리까지 반올림	`round(3.456, 1)`	`3.5`
`a // b, a % b`	몫, 나머지	`23 // 5, 23 % 5`	`4, 3`
`divmod(a, b)`	몫과 나머지를 한 번에 (튜플)	`divmod(23, 5)`	`(4, 3)`
`math.ceil(x)`	올림 (import math 필요)	`math.ceil(23 / 5)`	`5`

round(한국어): https://docs.python.org/ko/3.13/library/functions.html#round · math.ceil(한국어): https://docs.python.org/ko/3.13/library/math.html#math.ceil

round()

round(값, 자릿수) 는 값을 자릿수까지 반올림한다. 자릿수를 생략하면 정수로 만든다.

1) 기본 사용: 자릿수를 정한다

코드	결과	뜻
`round(3.14159, 2)`	`3.14`	소수 둘째 자리까지
`round(3.14159)`	`3`	자릿수 생략 → 정수
`round(123456, -3)`	`123000`	음수 자릿수 → 천 단위

2) 딱 중간(.5)이면 짝수 쪽으로 간다

코드	결과	엑셀이라면
`round(2.5)`	`2`	3
`round(3.5)`	`4`	4
`round(0.5)`	`0`	1

round()는 숫자를 지정한 자릿수에서 가장 가까운 값으로 반올림하는 함수이다.
반올림할 숫자가 두 결과의 정확한 중간값인 경우, 마지막 자리가 짝수인 값을 선택한다.
소수는 round(2.675, 2) → 2.67 처럼 예상과 다르게 나올 수도 있다.

round(값, 자릿수) → 자릿수까지 반올림. 정확히 .5 인 값만 짝수 쪽으로 가고, 그 외에는 보통 반올림과 같다

round()(한국어): https://docs.python.org/ko/3.13/library/functions.html#round

불리언과 비교·논리 연산

비교 연산자 (결과는 True / False)

연산자	의미	예시
>	크다	7 > 2 → True
>=	크거나 같다	7 >= 7 → True
<	작다	7 < 2 → False
<=	작거나 같다	2 <= 7 → True
==	같다	7 == 2 → False
!=	같지 않다	7 != 2 → True

논리 연산자 (여러 조건 묶기)

연산자	의미	예시
and	둘 다 참일 때 True	7 > 2 and 3 > 1 → True
or	하나만 참이어도 True	7 > 2 or 3 < 1 → True
not	참·거짓 뒤집기	not 7 > 2 → False

멤버십 검사 (값이 들어 있는지 확인)

연산자	의미	예시
in	값이 있으면 True	"서울" in ["서울", "부산"] → True
not in	값이 없으면 True	"제주" not in ["서울", "부산"] → True

비교·논리 연산(한국어): https://docs.python.org/ko/3.13/library/stdtypes.html#comparisons

불리언과 비교·논리 연산

```
# 비교·논리·멤버십 검사: 조건의 결과는 True 또는 False 다
sales = 150000
print(sales >= 100000)                    # 10만 이상인가 → True
print(sales > 100000 and sales < 200000)  # 두 조건 모두 참인가 → True
region = "서울"
print(region in ["서울", "부산"])          # 목록에 있는가 → True
print(not sales == 0)                     # 0이 아닌가 → True
# (not 은 참·거짓 뒤집기)
```

비교·논리 연산(한국어): https://docs.python.org/ko/3.13/library/stdtypes.html#comparisons

단락 평가: 앞에서 결론이 나면 뒤는 보지 않는다

and 는 앞이 거짓이면, or 는 앞이 참이면 결론이 이미 정해지므로 뒤 조건을 아예 확인하지 않는다.

and: 앞이 거짓이면 뒤를 건너뛴다

```
sales = []                          # 빈 리스트
if len(sales) > 0 and sum(sales) / len(sales) > 100:
    print("평균이 100을 넘는다")
# len(sales) > 0 이 False → 전체 False 확정 → 뒤 나눗셈은 실행 안 함
# → ZeroDivisionError 가 나지 않는다
```

or: 앞이 참이면 뒤를 건너뛴다

```
region = "서울"; sales = []       # 빈 리스트
if region == "서울" or sales[0] > 100:
    print("조건 만족")
# region == "서울" 이 True → 전체 True 확정 → sales[0] 은 실행 안 함
# → IndexError 가 나지 않는다
```

오류가 날 수 있는 조건은 뒤에, 그것을 막아 주는 조건은 앞에 쓴다. 순서를 바꾸면 오류가 난다

불리언 연산(한국어): https://docs.python.org/ko/3.13/reference/expressions.html#boolean-operations

단락 평가

앞 조건만 보고 결과가 정해지면, 뒤 조건(회색)은 실행되지 않는다.

and

`len(sales) > 0`

→ `False`

→ **전체 False**

```
sum(sales) / len(sales) > 100
```

나눗셈 안 함 → ZeroDivisionError 없음

or

`region == "서울"`

→ `True`

→ **전체 True**

```
sales[0] > 100
```

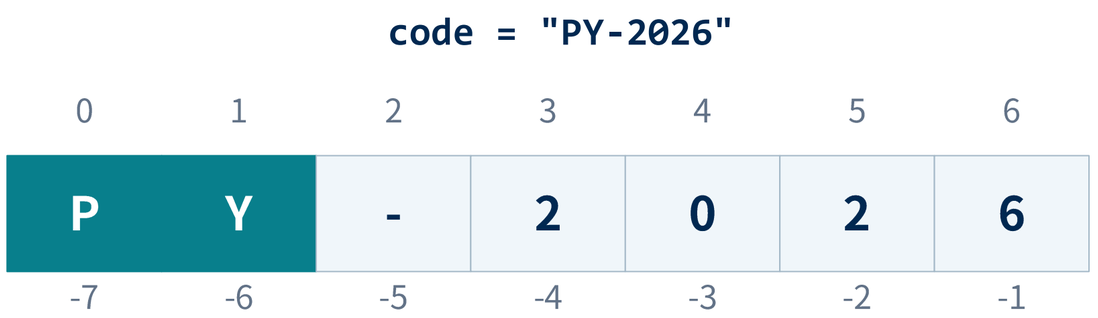
인덱싱 안 함 → IndexError 없음

불리언 연산(한국어): https://docs.python.org/ko/3.13/reference/expressions.html#boolean-operations

문자열과 리스트

문자열

- 문자열(str)은 따옴표로 감싼 글자의 나열이다



인덱싱과 슬라이싱 (일부 꺼내기)

문법	의미	예시
`[0]`	첫 번째 글자	`"PY-2026"[0] → "P"`
`[-1]`	마지막 글자	`"PY-2026"[-1] → "6"`
`[0:2]`	0번부터 2번 앞까지	`"PY-2026"[0:2] → "PY"`
`[3:]`	3번부터 끝까지	`"PY-2026"[3:] → "2026"`

`code[0:2]` → "PY" (0번부터 2번 앞까지)

`code[3:]` → "2026" (3번부터 끝까지)

`code[-1]` → "6" (뒤에서 첫 번째)

문자열 메서드(한국어): https://docs.python.org/ko/3.13/library/stdtypes.html#string-methods

주요 문자열 메서드

모두 새 문자열(또는 리스트)을 돌려주고 원래 문자열은 바뀌지 않는다.

메서드	의미	예시
`upper()`	대문자로 변환	`"py".upper() → "PY"`
`lower()`	소문자로 변환	`"PY".lower() → "py"`
`strip()`	양쪽 공백 제거	`" PY ".strip() → "PY"`
`replace()`	글자 바꾸기	`"PY-2026".replace("-", "_") → "PY_2026"`
`split()`	기준으로 나누기	`"PY-2026".split("-") → ['PY', '2026']`

문자열 메서드(한국어): https://docs.python.org/ko/3.13/library/stdtypes.html#string-methods

리스트

- 리스트(list)는 여러 값을 순서대로 담는 자료형이다
- 대괄호 [] 로 만들고, 인덱스로 꺼낸다

문법	의미	예시
`[]`	리스트 만들기	`sales = [100, 300, 200]`
`[0]`	인덱스로 꺼내기	`sales[0] → 100`
`append()`	끝에 값 추가	`sales.append(400)`

`sales = [100, 300, 200]`



`sales[0] → 100`, `sales[-1] → 200`
인덱스는 0부터, 뒤에서는 -1부터

리스트(한국어): https://docs.python.org/ko/3.13/tutorial/datastructures.html

리스트의 일부 선택과 변경

리스트는 여러 값을 담고, 필요한 구간을 꺼내거나 특정 값을 바꿀 수 있다.

```
sales = [100, 200, 300]
print(sales[0:2])     # [100, 200]: 끝 인덱스 2는 제외
sales[1] = 250        # 두 번째 값만 변경
sales.append(400)     # 마지막에 값 추가
print(sales)          # [100, 250, 300, 400]
```

sales[0:2] → 새 리스트



sales 자체가 바뀐다



1) sales[0:2] 는 일부 값을 담은 새 리스트를 만든다

2) sales[1] = 250 과 append() 는 sales 자체를 바꾼다

리스트(한국어): https://docs.python.org/ko/3.13/tutorial/datastructures.html

리스트에서 값 지우기: remove · pop · del

방법	기준	예 (sales = [100, 300, 200])	결과
`sales.remove(300)`	값을 알 때 (첫 번째 하나만)	`sales.remove(300)`	`[100, 200]`
`sales.pop(0)`	위치를 알고, 지운 값을 쓰고 싶을 때	`first = sales.pop(0)`	`first = 100,` `sales = [300, 200]`
`del sales[0]`	위치를 알고, 값은 필요 없을 때	`del sales[0]`	`[300, 200]`

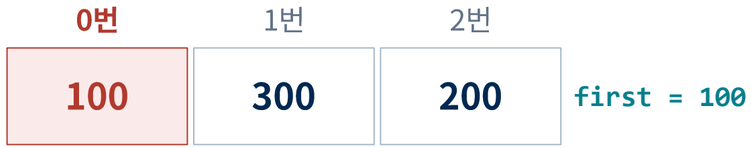
값으로 지우기

`sales.remove(300)`



위치로 지우기 + 지운 값 받기

`first = sales.pop(0)`



위치로 지우기

`del sales[0]`



없는 값을 remove 하면 ValueError, 없는 위치를 pop·del 하면 IndexError 가 난다

리스트 메서드(한국어): https://docs.python.org/ko/3.13/tutorial/datastructures.html#more-on-lists

sort() 와 sorted()

주의: sort() 와 sorted() 의 차이

sort() 는 리스트 자체를 바꾸고 None 을 돌려준다
sorted() 는 원본은 그대로 두고 정렬된 새 리스트를 돌려준다
x = x.sort() 라고 쓰면 x 가 None 이 된다

```
sales = [300, 100, 200]
sales.sort()             # 올바른 사용: sales 가 [100, 200, 300] 으로 바뀐다
sales = sales.sort()     # 잘못된 사용: sales 가 None 이 된다
```

sorted(sales) 원본은 그대로, 새 리스트 반환



sales.sort() 원본이 바뀌고 None 반환



리스트(한국어): https://docs.python.org/ko/3.13/tutorial/datastructures.html · sorted(한국어): https://docs.python.org/ko/3.13/library/functions.html#sorted

오류 확인 실습

NameError

정의하지 않은 이름을 썼다

변수를 만든 셀을 먼저 실행했는지, 철자가 맞는지 확인

```
print(totl)
```

TypeError

자료형이 맞지 않는 연산

"12" + 3 처럼 문자열과 숫자를 더했는지 확인

```
"12" + 3
```

IndexError

리스트 범위 밖 인덱스

len() 으로 길이 확인, 인덱스는 0부터

```
[10, 20, 30][3]
```

ZeroDivisionError

0 으로 나눔

빈 리스트 평균. 나누기 전에 개수 확인

```
10 / 0
```

IndentationError

들여쓰기 오류

같은 블록은 같은 칸만큼 들여쓴다 (공백 4칸이 관례)

```
    x = 1   # 맨 앞 공백
```

내장 예외(한국어): https://docs.python.org/ko/3.13/library/exceptions.html · PEP 8 들여쓰기: https://peps.python.org/pep-0008/#indentation